

ISelec 3 Reviewer

Linear Regression

Linear Regression is a supervised machine learning algorithm used to predict continuous values based on relationships between variables.

Linear Regression Function

$$y = a + bx$$

Slope (b) of Regression Line

$$b = r \frac{S_y}{S_x}$$

Y-Intercept (a) of Regression Line

$$a = \bar{y} - b\bar{x}$$

No.	Assignment Score	Exam Score	x-x'	y-y'	(x-x')*(y-y')	(x-x')^2	(y-y')^2
1	12	50	-44.700	-18.900	844.830	1998.090	357.210
2	23	55	-33.700	-13.900	468.430	1135.690	193.210
3	31	58	-25.700	-10.900	280.130	660.490	118.810
4	45	60	-11.700	-8.900	104.130	136.890	79.210
5	53	72	-3.700	3.100	-11.470	13.690	9.610
6	62	69	5.300	0.100	0.530	28.090	0.010
7	71	78	14.300	9.100	130.130	204.490	82.810
8	84	74	27.300	5.100	139.230	745.290	26.010
9	89	85	32.300	16.100	520.030	1043.290	259.210
10	97	88	40.300	19.100	769.730	1624.090	364.810
	56.700	68.900			3245.700	7590.100	1490.900
r	0.965	b	0.428				
Sy	12.871	a	44.654				
Sx	29.040	y	when x=25	25	55.344		
			when x=65	65	72.449		
			when x=80	80	78.864		

PEARSON CORRELATION COEFFICIENT (r)

$$r = \frac{\sum((x-\bar{x})(y-\bar{y}))}{\sqrt{\sum(x-\bar{x})^2 \sum(y-\bar{y})^2}}$$

Handwritten calculations for Linear Regression:

$r = 0.965$

$b = r \frac{S_y}{S_x}$
 $= 0.965 \times \frac{12.871}{29.040}$
 $= 0.428$

$a = \bar{y} - b\bar{x}$
 $= 68.900 - 0.428 \times 56.700$
 $= 68.900 - 24.268$
 $a = 44.632 \text{ or } 44.654$

$y = a + bx$
 $= 44.654 + 0.428 \times 25$
 $= 55.344$

K-MEANS – an algorithm that group data path based on K (clusters)

CLUSTERING- process of grouping similar objects into clusters

CENTROID- the center point of a cluster

EUCLIDEAN DISTANCE

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Centroid - the center point of a cluster

Euclidean Distance

$$d(P_i, C_k) = \sqrt{(X_i - X_k)^2 + (Y_i - Y_k)^2}$$

$$d = \sqrt{(2-2)^2 + (10-5)^2} = \sqrt{0^2 + 5^2} = 5.000$$

Data Points	x	y	Distance to	Cluster	New cluster
2	10	0.000			
2	5	5.000			
8	4				

Centroids

Cluster 1
 $(8+5+7+6+4)/5 = 6$
 $(9+8+5+4+7)/5 = 6$

Cluster 2
 $(2+1)/2 = 1.5$
 $(5+2)/2 = 3.5$

Initial Centroids
 $(2, 10)$
 $(5, 8)$
 $(1, 2)$

2nd iteration

Data Points	x	y	Distance to	Cluster	New cluster
2	10	0.000	5.657	1	1
2	5	5.000	4.123	1	1
8	4	6.495	2.828	3	3
5	8	3.606	2.236	2	2
7	5	7.071	1.414	5.701	2
6	4	7.211	2.000	4.536	2
1	2	8.062	6.403	1.581	3
4	9	2.236	3.606	6.042	2

Centroids
 $(2, 10)$
 $(6, 6)$
 $(1.5, 3.5)$

3rd iteration

Data Points	x	y	Distance to	Cluster	New cluster
2	10	1.118	6.539	6.519	1
2	5	4.610	4.602	1.581	3
8	4	7.433	1.953	6.519	2
5	8	2.500	2.132	5.701	2
7	5	6.021	0.559	5.701	2
6	4	6.265	1.346	4.528	2
2	7	7.262	6.338	1.581	3
9	1	1.118	4.507	6.042	1

Centroids
 $1: (2+4)/2 = 3$
 $(10+9)/2 = 9.5$
 $2: (4+5+7+6)/4 = 6.5$
 $(7+8+5+4)/4 = 5.25$
 $3: (2+1)/2 = 1.5$
 $(5+2)/2 = 3.5$

Distance to

Data Points	547	9	7	4,338	1.5	3.5	Cluster	New cluster
2	10	1.944	7.557	6.519	1	1	3	3
2	5	4.333	5.044	1.581	2	2	2	2
8	4	6.466	1.555	0.519	1	1	2	2
5	8	1.666	4.177	5.701	2	2	2	2
7	5	6.666	0.667	5.701	2	2	2	2
6	4	5.518	1.054	4.528	2	2	2	2
1	2	7.411	6.955	1.581	3	3	3	3
4	9	8.333	5.548	6.042	1	1	1	1

Clusters

- (2,10), (5,8), (4,9)
- (8,4), (7,5), (6,4)
- (3,5), (1,2)

1. $(2+5+4)/3 = 3.667$
 $(10+8+9)/3 = 9$
2. $(8+7+6)/3 = 7$
 $(4+5+4)/3 = 4.333$
3. $(2+1)/2 = 1.5$
 $(5+2)/2 = 3.5$

PHYTON PROGRAMMING

– is a high-level programming language developed by Guido Van Rossum

-officially released last 1991

DATA TYPE IN PYTHON

Int, float, complex, str, bool, list, tuple, dict, set, None Type

ARITHMETIC OPERATIONS

+, -, *, /, % (modulo division), ** (exponentiation)

BACKLASH CODE/ ESCAPE SEQUENCE

\n new line

\t tab space

\\

\\

\"

\b backspace

```
print("Hello World")
print("Hello \n World")
print("Hello \\ World")
print("Hello / World/")
print("Hello \" World \"")
print("Hello World\b")
```

```
a = 2
b = 3
c = 4.0
d = 5.0
e = TRUE
print(a * c)

Set = "BGS 3A"

a = 2
b = 10
Print(a + b)
Print(Set)
type(a)
type(b)
type(Set)
```

Looping Structures

```
# For loop
for i in range(5):
    print(i)

# While loop
x = 0
while x < 5:
    print(x)
    x += 1
```

Branching Structures

```
# Match (Equivalent of switch)
```

Branching Structures

```
# Match (Equivalent of switch)
Operator = input("Enter an operator")
match operator:
    case "+":
        print("For addition")
    case "-":
        print("For subtraction")
    case "*":
        print("For multiplication")
    case "/":
        print("For division")
    case "%":
        print("For modulo division")
    case _:
        print("Invalid Operator")
```



```
# Using if-else if
operator = input("Enter an operator")
if (operator == "+"):
    print("For addition")
elif (operator == "-"):
    print("For subtraction")
elif (operator == "*"):
    print("For multiplication")
elif (operator == "/"):
    print("For division")
elif (operator == "%"):
    print("For modulo division")
else:
    print("Invalid Operator")
```

```
age = 18
if (age >= 18):
    print("Adult")
else:
    print("Minor")

# Asking for input (String)
# Example
name = input("Enter your name")
print("Hello", name)

# Note: you have to typecast if any other
# datatype is used. Example (Integer)
x = int(input("Enter a number"))
```

1. Make a program that will ask for a number.
If the number is even, display 'The number is even', otherwise display 'The number is odd'.

2. Using the print command, display the ff. lines

Fruit	Quantity	Price
Papaya	3	35.00/kg
Banana	5	50.00/kg
Guava	4	25.00/kg

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("The number is even")
else:
    print("The number is odd")

Enter a number: 30
The number is even
```

```
print("""\tList of Fruits
Fruits  Quantity  Price
Papaya   3         35.00/kg
Banana   5         50.00/kg
Guava    4         25.00/kg
""")
```

Fruits	Quantity	Price
Papaya	3	35.00/kg
Banana	5	50.00/kg
Guava	4	25.00/kg

1. What will be the output of the following Python code snippet?

```
x = [1, 2, 3]
y = x
y.append(4)
print(x)
```

ANS. [1, 2, 3, 4]

2. Which of the following is the correct way to define a function in Python?

ANS. Def my_func():

3. What is the primary difference between a list and a tuple in Python?

ANS. List are mutable, while tuples are immutable.

4. What will be the result of executing `bool("")` in Python?

ANS. False

5. How do you start a multi-line comment in Python?

ANS. Python does not have a formal multi-line comment syntax; you use multiple `#` symbols or unassigned multi-line strings

6. What does the `len()` function return when passed a dictionary?

ANS. The total number of keys in the dictionary

7. What is the output of `print(2 ** 3)` in Python? (exponentiation)

ANS. 8

8. Which keyword is used to handle exceptions or errors in Python?

ANS. Except

9. What will `"hello".upper()` return?

ANS. HELLO

10. What is the purpose of the `break` statement in a loop?

ANS. To terminate the loop completely and resume execution at the next statement outside the loop.

CONFUSION MATRIX- A Confusion Matrix is used to evaluate classification models.

It compares:

- Actual values
- Predicted values

Structure of Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Meaning

- **TP (True Positive)**
Correctly predicted positive
- **TN (True Negative)**
Correctly predicted negative
- **FP (False Positive)**
Incorrectly predicted positive
- **FN (False Negative)**
Incorrectly predicted negative

Example

	Predicted Yes	Predicted No
Actual Yes	40	10
Actual No	5	45

Here:

- TP = 40
- FN = 10
- FP = 5
- TN = 45

PERFORMANCE METRICS (ACCURACY, PRECISION, RECALL, F1-SCORE)

These metrics measure how good a classification model performs.

Accuracy

Formula

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Meaning

Percentage of correct predictions.

Example

If 90 out of 100 predictions are correct:

- Accuracy = 90%

Precision

Formula

$$Precision = \frac{TP}{TP+FP}$$

Meaning

Measures how many predicted positives are actually correct.

High Precision

Few false positives.

Recall

Formula

$$Recall = \frac{TP}{TP+FN}$$

Meaning

Measures how many actual positives are correctly identified.

High Recall

Few false negatives.

F1-Score

Formula

$$F1 = 2 \left(\frac{Precision \times Recall}{Precision + Recall} \right)$$

Meaning

Balance between Precision and Recall.

Use Case

Useful when dataset is imbalanced.

Quick Comparison Table

Metric	Focus
Accuracy	Overall correctness
Precision	Correct positive predictions
Recall	Finding all positives
F1-score	Balance of precision & recall